

Principle Violations

1. Missing Dependency Injection Registration

Principle violated: Dependency Inversion Principle

Location: Program.cs

Why it is a violation: The ItemController correctly depends on the IItemReader abstraction (this is true for now, but, after adding the service layer, it will depend on IGradeService), and ItemRepository correctly implements it. However, inside Program.cs, the binding between IItemReader and ItemRepository is never registered in the DI container. This violates the Dependency Inversion Principle, because the composition root, Program.cs, is responsible for wiring abstractions to their concrete implementation.

Fix applied: Added the service registration inside Program.cs.

2. Inheritance-based Extension

Principle violated: Open-Closed Principle

Location: ItemRepository.cs

Why it is a violation: ItemRepository.cs exposes protected fields and virtual methods, meaning it is designed to be extended through inheritance, which violates Open-Closed principle, because it invites subclasses to directly access and override the internal state (`_items`, `_nextId`). The Open-Closed principle states that a class should be open for extension, but closed for modification. The better way to achieve this is through a new interface implementation or composition. The protected fields also violate the Encapsulation principle, which, even though it is not a SOLID principle, is a fundamental part of OOP.

Fix applied: Replaced protected with private and removed the virtual keyword.

3. Logic in controller, endpoint with multiple responsibilities, absence of Service

Principle violated: Single Responsibility Principle

Location: ItemController.cs, GetAll() method

Why it is a violation: The controller (ItemController.cs) has multiple responsibilities: handling HTTP requests and also doing business logic. This breaks the Single Responsibility Principle. The GetAll() endpoint returns both the list of grades and the statistics, which also breaks the Single Responsibility Principle.

Fix applied: Introduced a service layer (GradeService) and moved all business logic there. Split the endpoint into two: `api/grade` returns only the list of grades, and `api/grade/statistics` returns only the statistics.

Additional Improvements

The names of the classes, using the word Item, are too generic for this application. As such, I renamed them to use the word Grade: GradeController.cs, IGradeReader.cs, Grade.cs and GradeRepository.cs. I have also modified the `launchUrl` to `api/grade` in `launchSettings.json`, and renamed the appropriate occurrences of “item” in variable names to “grade”.

I kept the filtering for the IsActive attribute of the grades inside the repositories. I considered it as a flag that marks if a record was deleted or not, and, under this interpretation, it is more appropriate to filter it on the repository layer.